

# Intent Document

---

## Intent profile

Status: complete

Artifact: `90e7970c-0d00-4d2e-b307-c5cb6868326d`

## Vision

Modernize the legacy refund and fee approval service through a behavior-preserving Python 3 rewrite that modularizes the implementation, documents implicit contracts, and adds regression tests while preserving observed settlement-console routes, request/response behavior, refund approval rules, fee calculations, audit semantics, status codes, and operational assumptions.

## Target personas

- Settlement Console Operator: submits refund approvals through legacy settlement workflows and needs identical approved/denied responses without client disruption.
- Finance Administrator: relies on the existing finance\_admin role field behavior for high-value refund approval outcomes and needs threshold logic preserved exactly.
- Audit or Compliance Reviewer: queries refund audit records and needs returned audit records, filters, ordering, and field names to remain compatible with legacy behavior.
- Branch Office Support Operator: runs the lightweight service in constrained branch environments and needs dependency-light Python 3 operation with familiar CLI/environment configuration.
- Modernization Engineer: refactors and tests the service while using execution against the legacy implementation as the final parity authority.

## Core features

- Legacy behavior parity as primary goal (priority 1)
- Refund approval behavior preservation (priority 2)
- Threshold and date exception preservation (priority 3)
- Fee quote behavior preservation (priority 4)
- Audit persistence and lookup behavior preservation (priority 5)
- Health endpoint preservation (priority 6)
- Python 3 standardization (priority 7)
- Modular service structure (priority 8)
- Explicit compatibility contracts (priority 9)
- Behavior-preserving tests and basic diagnostics (priority 10)

## Technical constraints

- This is a behavior-preserving rewrite, not a product expansion.
- Legacy observed behavior is authoritative, and execution against the legacy service determines parity.
- Do not add authentication, authorization, RBAC, identity middleware, bearer-token checks, OIDC/JWT handling, mTLS identity, permissions, policy middleware, or deny-by-default endpoint security.
- Do not add CI/CD requirements.

- Do not change existing routes, request fields, response fields, HTTP status codes, JSON response shapes, default route behavior, or validation error semantics unless proven equivalent by legacy execution.
- Do not change audit record semantics, SQLite audit ordering behavior, audit write timing, or audit lookup behavior.
- Do not change threshold logic, last-business-day exception behavior, fee truncation behavior, or negative amount handling.
- Do not introduce new product features or new infrastructure requirements.
- Allowed modernization is limited to Python 3 standardization, modularization, explicit contracts, behavior-preserving tests, documentation, and basic diagnostics that do not alter existing API behavior.
- Primary language remains Python.
- Current implementation is a single-file legacy service at legacy/server.py.
- Current implementation uses Python standard-library HTTP, JSON, logging, CLI parsing, threading, URL parsing, and SQLite components.
- Dependency additions should remain minimal or avoided because branch offices historically run the service with system Python.
- Current routes must be preserved: GET /health, GET /audit, POST /refunds/approve, and POST /fees/quote.
- Current maximum request body size remains MAX\_BODY = 65536 bytes unless legacy execution proves equivalent behavior.
- SQLite remains the audit data store unless a separately approved versioned change is made.
- Current database default behavior uses REFUND\_DB or --database, falling back to :memory:.
- Current bind, port, database, and debug configuration behavior must remain compatible unless explicitly approved as a versioned operational change.
- Refund approval validation and posting must preserve legacy ordering so audit writes occur only after validation and approval logic.
- Fee calculation must preserve legacy float math and display formatting compatibility with the nightly settlement export.

## Confidence

Overall: 94%

> The provided code and requested regeneration constraints give a complete modernization intent centered on legacy parity.

| Section               | Score | Why                                                                                                                                                             | How to Improve                                                                                                                             |
|-----------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Vision                | 96%   | The modernization goal and behavior-preserving constraints are explicitly stated and strongly supported by the legacy service code.                             | Add measurable migration success criteria such as supported Python 3 version, parity test pass threshold, and rollout acceptance process.  |
| Target personas       | 86%   | Personas are inferred from routes, business roles, audit lookup, operational comments, and modernization responsibilities rather than direct user research.     | Confirm which legacy clients and branch office users are still active and whether any external systems call these endpoints directly.      |
| Core features         | 97%   | Endpoint behavior, business rules, validation rules, audit semantics, and allowed modernization scope are clearly evidenced in the source and user constraints. | Provide the original numbered requirement source for REQ-001, REQ-007, REQ-014, REQ-022, and REQ-031 to verify labels and wording exactly. |
| Technical constraints | 96%   | The input provides explicit binding constraints plus concrete implementation details for runtime, routes, SQLite, standard-library usage, and excluded changes. | Confirm the target Python 3 version and whether dependency-free operation is mandatory or simply preferred.                                |

---

# Modernization Analysis

> Review the proposals below and select the modernization areas you want to pursue. You can reply in the chat to confirm your choices or ask follow-up questions.

## Tech Stack

Language, framework, and tooling modernization

| # | Area                                     | Current                                                                                                                                                                                                                               | Proposed                                                                                                                                                                                                                                       | Rationale                                                                                                                                                   |
|---|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Modernization scope and parity authority | Legacy behavior is embedded in <code>`legacy/server.py`</code> and is not formally separated from modernization goals.                                                                                                                | Treat this as a behavior-preserving rewrite where execution against the legacy service is the final parity authority, including REQ-001, REQ-007, REQ-014, REQ-022, and REQ-031.                                                               | This prevents accidental product expansion or security/infrastructure changes and makes observed legacy behavior the compatibility contract.                |
| 2 | Python runtime                           | Single Python file supports Python 2 and Python 3 through compatibility imports and <code>`basestring`</code> fallback; no supported runtime is declared.                                                                             | Standardize on a supported Python 3 runtime such as Python 3.14 and declare it in <code>`pyproject.toml`</code> or equivalent project metadata.                                                                                                | Python 2 is EOL, and removing compatibility branches after parity tests reduces legacy complexity without changing behavior.                                |
| 3 | HTTP transport                           | <code>`BaseHTTPRequestHandler`</code> , <code>`HTTPServer`</code> , and <code>`ThreadingMixIn`</code> handle routing, request parsing, JSON responses, and errors directly.                                                           | Keep the Python standard-library HTTP model using Python 3 modules <code>`http.server`</code> , <code>`socketserver`</code> , and <code>`urllib.parse`</code> , preserving all routes and response behavior.                                   | This respects the dependency-light constraint and avoids framework-driven contract drift.                                                                   |
| 4 | Service structure                        | <code>`legacy/server.py`</code> mixes transport, validation, approval policy, fee logic, audit persistence, CLI parsing, and runtime lifecycle.                                                                                       | Split into behavior-preserving modules such as <code>`transport`</code> , <code>`validation`</code> , <code>`approval_policy`</code> , <code>`fees`</code> , <code>`audit_store`</code> , <code>`config`</code> , and <code>`runtime`</code> . | Modularization lowers cognitive load while keeping existing logic, routes, status codes, JSON shapes, and persistence semantics unchanged.                  |
| 5 | Request and response contracts           | Contracts are implicit in helper functions including <code>`cleanText`</code> , <code>`readAmount`</code> , <code>`parseWhen`</code> , <code>`normalizeRoles`</code> , <code>`approve_refund`</code> , and <code>`quote_fee`</code> . | Document explicit compatibility contracts for <code>`/health`</code> , <code>`/audit`</code> , <code>`/refunds/approve`</code> , and <code>`/fees/quote`</code> based on observed legacy execution.                                            | Contracts make legacy behavior reviewable and testable without introducing new validation rules or schema changes.                                          |
| 6 | Testing approach                         | No test files are evidenced; edge cases and business rules are only represented in implementation code.                                                                                                                               | Add pytest characterization and regression tests that exercise the legacy and rewritten services for equivalent requests.                                                                                                                      | Golden-master style parity testing protects refund approval, audit, fee truncation, negative amount, date exception, and error behavior during the rewrite. |
| 7 | Approval rules                           | <code>`approve_refund`</code> enforces threshold, role-list normalization, last-business-day bypass, negative amount effective value handling, and audit insertion.                                                                   | Extract approval policy into a dedicated module while preserving REQ-007 and REQ-014 exactly.                                                                                                                                                  | Separating the policy improves maintainability while preserving the authoritative legacy decision logic.                                                    |
| 8 | Fee calculation                          | <code>`quote_fee`</code> uses float math, clamps negative amounts to <code>`0.0`</code> , truncates with <code>`int((amount * 0.025) * 100) / 100.0`</code> , and formats with two decimals.                                          | Extract fee calculation into a dedicated module while preserving REQ-022 exactly.                                                                                                                                                              | The calculation is financially sensitive and must remain byte-for-byte compatible in observable responses.                                                  |

| # | Area           | Current                                                                                                                                                         | Proposed                                                                                                                        | Rationale                                                                                                                      |
|---|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 9 | Audit behavior | Approved refunds insert one SQLite audit row; denied refunds do not; <code>`/audit`</code> returns all or filtered records ordered by <code>`audit_id`</code> . | Encapsulate SQLite audit access behind an internal <code>`AuditStore`</code> module while preserving REQ-031 semantics exactly. | This improves testability without changing audit write timing, fields, action value, timestamp source, filtering, or ordering. |

## Packages & Dependencies

### Dependency upgrades and vulnerability remediation

| # | Area                                 | Current                                                                                                                                             | Proposed                                                                                                                                                                         | Rationale                                                                                                            |
|---|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1 | Runtime dependencies                 | The service intentionally uses only Python standard-library modules and has no <code>`requirements.txt`</code> or dependency manifest.              | Preserve zero external runtime dependencies and document that constraint in <code>`pyproject.toml`</code> or project documentation.                                              | Keeping runtime dependency-free supports branch-office operation and avoids introducing new deployment requirements. |
| 2 | Python standard-library HTTP modules | Python 2 modules <code>`BaseHTTPServer`</code> , <code>`SocketServer`</code> , and <code>`urlparse`</code> are attempted before Python 3 fallbacks. | Use Python 3 standard-library modules <code>`http.server`</code> , <code>`socketserver`</code> , and <code>`urllib.parse`</code> after parity tests pass.                        | This removes obsolete compatibility paths while preserving the current standard-library architecture.                |
| 3 | SQLite access                        | The service uses Python standard-library <code>`sqlite3`</code> directly through a module-global connection and <code>`threading.RLock`</code> .    | Continue using standard-library <code>`sqlite3`</code> , but wrap it in a small internal audit-store module with the same connection, lock, timeout, schema, and query behavior. | This avoids adding an ORM or new database dependency while improving isolation for tests.                            |
| 4 | Testing package                      | No test framework is evidenced.                                                                                                                     | Add <code>`pytest`</code> as a development-only dependency, pinned to a stable major such as <code>`pytest&gt;=8,&lt;9`</code> unless project policy requires stricter pinning.  | Pytest supports concise characterization tests while adding no runtime dependency to the deployed service.           |
| 5 | Contract fixtures                    | No schema files, golden responses, or request fixtures are evidenced.                                                                               | Add repository-local JSON fixtures for known legacy requests and responses rather than introducing a runtime schema package.                                                     | Fixtures document observed behavior and reduce the chance of accidental response-shape drift.                        |

## Infrastructure

### Hosting, orchestration, and platform modernization

| # | Area                  | Current                                                                                                                                                                | Proposed                                                                                                                           | Rationale                                                                                                              |
|---|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1 | Deployment model      | Standalone threaded Python HTTP service started by CLI with <code>`--bind`</code> , <code>`--port`</code> , <code>`--database`</code> , and <code>`--debug`</code> .   | Preserve the same standalone startup model and CLI/environment configuration surface.                                              | The rewrite must not introduce containers, orchestrators, managed platforms, or other new infrastructure requirements. |
| 2 | Runtime configuration | <code>`PORT`</code> and <code>`REFUND_DB`</code> environment variables are supported, with defaults of port <code>`8080`</code> and database <code>`:memory:`</code> . | Preserve existing defaults and document their behavior, including the non-durable nature of <code>`:memory:`</code> audit storage. | Documentation improves operator clarity without changing startup behavior or audit semantics.                          |
| 3 | Network binding       | Default bind address is <code>`0.0.0.0`</code> , with override through <code>`--bind`</code> .                                                                         | Preserve the current bind default and override behavior, while documenting operational assumptions and risks.                      | Changing the default bind could break legacy deployments, so the behavior-preserving                                   |

| # | Area            | Current                                                                                                                                             | Proposed                                                                                                                                                                      | Rationale                                                                                                            |
|---|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
|   |                 |                                                                                                                                                     |                                                                                                                                                                               | rewrite should not alter it.                                                                                         |
| 4 | Health endpoint | <code>`GET /health`</code> returns static JSON <code>`{"service":"refund-fee","status":"ok"}`</code> with HTTP 200.                                 | Preserve the exact route, status code, and JSON response shape.                                                                                                               | Health behavior may be consumed by legacy scripts or monitors and must remain compatible.                            |
| 5 | Diagnostics     | Standard-library logging records requests through <code>`log_message`</code> and startup through <code>`run`</code> ; debug mode adjusts log level. | Keep standard-library logging and add only behavior-neutral diagnostic clarification, such as clearer startup logs for configured bind, port, database mode, and debug state. | Basic diagnostics help support the rewrite without changing APIs, persistence semantics, or deployment requirements. |
| 6 | CI/CD           | No CI/CD configuration is evidenced.                                                                                                                | Do not add CI/CD requirements as part of this modernization intent.                                                                                                           | This follows the binding constraint to avoid new delivery infrastructure requirements.                               |

## Technical Capabilities

New capabilities unlocked by modernization

| # | Area                                       | Current                                                                                                                                                                               | Proposed                                                                                                                                           | Rationale                                                                                                                  |
|---|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| 1 | Behavioral parity verification             | Compatibility is inferred manually from the single source file.                                                                                                                       | Use legacy-service execution as the authoritative oracle for parity tests across routes, status codes, JSON bodies, and side effects.              | This operationalizes REQ-001 and makes compatibility measurable.                                                           |
| 2 | Refund approval compatibility              | REQ-007 behavior is implemented directly in <code>`approve_refund`</code> , including required fields, optional fields, validation, response fields, and status behavior.             | Codify REQ-007 in documentation and characterization tests without changing endpoint behavior.                                                     | This protects the settlement-console approval contract during modularization.                                              |
| 3 | Threshold and date exception compatibility | REQ-014 logic uses <code>`AUTH_LIMIT = 500`</code> , <code>`int(effective) &gt; AUTH_LIMIT`</code> , <code>`finance_admin`</code> , and last-business-day bypass.                     | Capture REQ-014 as explicit tests for below-threshold, above-threshold, finance-admin-present, finance-admin-missing, and last-business-day cases. | These rules drive approval outcomes and are high-risk if reimplemented informally.                                         |
| 4 | Fee calculation compatibility              | REQ-022 behavior uses legacy float math, truncation, two-decimal display formatting, <code>`USD`</code> , and negative amount normalization.                                          | Lock REQ-022 with regression tests using representative decimal, large, zero, and negative amounts.                                                | Fee results must remain settlement-compatible even where decimal arithmetic might appear more modern.                      |
| 5 | Audit compatibility                        | REQ-031 behavior writes audit only for approved refunds and returns audit records as <code>`{"records":[...]}`</code> ordered by <code>`audit_id`</code> .                            | Test audit insertion, denial no-write behavior, timestamp source, action value, unfiltered lookup, filtered lookup, and ordering.                  | Audit semantics are a key observable side effect and must survive refactoring unchanged.                                   |
| 6 | Legacy route compatibility                 | Routes are <code>`/health`</code> , <code>`/audit`</code> , <code>`/refunds/approve`</code> , and <code>`/fees/quote`</code> ; unknown routes return <code>`404`</code> problem JSON. | Add route-level tests covering existing paths, methods, unknown routes, and existing problem response shapes.                                      | This prevents accidental routing or status-code drift during transport refactoring.                                        |
| 7 | Validation behavior clarity                | Bad request behavior is spread across body parsing, text cleaning, amount parsing, date parsing, and role normalization helpers.                                                      | Document and test existing validation messages and <code>`400`</code> error shapes as observed from legacy execution.                              | Legacy clients may depend on specific failure behavior, so validation must be preserved unless explicitly versioned later. |
| 8 | Maintainability                            | All behavior lives in one file, increasing context switching for                                                                                                                      | Enable isolated maintenance of transport, validation, policy, fees,                                                                                | The rewrite improves internal clarity while                                                                                |

| # | Area | Current        | Proposed                                                           | Rationale                                            |
|---|------|----------------|--------------------------------------------------------------------|------------------------------------------------------|
|   |      | small changes. | audit store, and runtime code through behavior-preserving modules. | keeping the external API and side effects unchanged. |

## Risk Register

Top risks for this modernization effort

| # | Area                  | Risk                                                                                                                                 | Likelihood | Impact | Mitigation                                                                                                                                         |
|---|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------|------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Behavior parity       | The rewrite may unintentionally change status codes, JSON key ordering, response shapes, validation messages, or route behavior.     | high       | high   | Run golden-master tests against the legacy service and treat legacy execution as the final authority for parity.                                   |
| 2 | Refund approval rules | Threshold, `finance_admin`, last-business-day, role normalization, or negative amount behavior could be reimplemented incorrectly.   | medium     | high   | Implement REQ-007 and REQ-014 as mandatory characterization tests before refactoring approval code.                                                |
| 3 | Fee calculation       | A developer may replace legacy float truncation with rounded decimal arithmetic, changing settlement-visible fee values.             | medium     | high   | Preserve the exact REQ-022 formula and add regression cases for values that distinguish truncation from rounding.                                  |
| 4 | Audit semantics       | Modularizing persistence may alter audit write timing, denied-refund no-write behavior, timestamp source, filtering, or ordering.    | medium     | high   | Create audit parity tests for approved writes, denied no-writes, `refund.approved`, submitted timestamp usage, filtering, and `audit_id` ordering. |
| 5 | Python 3 migration    | Bytes/text handling, URL parsing, JSON decoding, or standard-library HTTP behavior may differ after removing Python 2 compatibility. | medium     | medium | Migrate one boundary at a time and compare legacy and rewritten HTTP responses for valid, malformed, and edge-case requests.                       |